

Отчёт по задаче о китайском почтальоне (неориентированный взвешенный граф)

Автор: Речкин Вадим Сергеевич

Группа: 23Б08-мм

GitHub: github.com/VadosikRRR/.../postman.cpp

1. Формулировка задачи

Дан связный неориентированный взвешенный граф $G = (V, E)$. Каждое ребро имеет неотрицательный вес (длину).

Требуется найти минимальную длину **замкнутого** маршрута, который проходит по каждому ребру графа хотя бы один раз.

Формат входных данных

- Первая строка: n m — число вершин и число рёбер.
- Далее m строк: u v w — ребро между вершинами u и v веса w .
- Вершины нумеруются от 1 до n .

Формат выходных данных

Одно число — минимальная длина искомого замкнутого маршрута.

2. Пояснение к задаче и подробная идея решения

2.1. Почему важны чётные и нечётные вершины

Степень вершины — это количество рёбер, инцидентных этой вершине.

Для существования эйлера цикла в связном неориентированном графе необходимо и достаточно, чтобы степени всех вершин были чётными.

- Если все вершины чётные, тогда можно пройти каждое ребро ровно один раз, и ответ равен сумме весов всех рёбер.
- Если есть вершины нечётной степени, эйлеров цикл отсутствует. Чтобы получить замкнутый маршрут, нужно продублировать некоторые рёбра/пути.

2.2. Как получить минимальную добавку к сумме рёбер

Обозначим через S сумму весов всех рёбер графа. Любой корректный маршрут обязан пройти каждое ребро хотя бы один раз, поэтому базовая часть ответа всегда равна S .

Остаётся минимизировать добавку, возникающую из-за нечётных вершин:

1. Найти все вершины нечётной степени и собрать их в множество O .
2. Для любых двух вершин графа вычислить кратчайшие расстояния (алгоритм Флойда–Уоршелла).
3. Среди вершин множества O найти минимальное по стоимости попарное разбиение.

Если вершины $a, b \in O$ объединяются в пару, мы как бы добавляем в маршрут кратчайший путь между a и b . Это меняет чётность обеих вершин, и после попарного соединения всех нечётных вершин граф становится эйлеровым.

2.3. DP по битовой маске

Пусть $k = |O|$ (это число всегда чётно).

Введём:

$dp[mask]$ = минимальная стоимость попаривания уже выбранных вершин из O .

Здесь ‘mask’ — битовая маска длины k , где бит равен 1, если соответствующая нечётная вершина уже использована в паре.

Переход:

- выбираем первую неиспользованную вершину i ,
- пробуем объединить её с любой другой неиспользованной вершиной j ,
- обновляем

$$dp[newMask] = \min(dp[newMask], dp[mask] + dist[O_i][O_j]).$$

Итог:

$$\text{Ответ} = S + dp[(1 \ll k) - 1].$$

3. Код программы

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    ifstream fin("example_input.txt");
    int n, m;
    fin >> n >> m;

    const int INF = 1000000000;
```

```

vector<vector<int>> dist(n, vector<int>(n, INF));
vector<int> degree(n, 0);

for (int i = 0; i < n; ++i) {
    dist[i][i] = 0;
}

int sumEdges = 0;

for (int i = 0; i < m; ++i) {
    int u, v;
    int w;
    fin >> u >> v >> w;
    --u;
    --v;

    sumEdges += w;
    degree[u]++;
    degree[v]++;

    if (w < dist[u][v]) {
        dist[u][v] = w;
        dist[v][u] = w;
    }
}

for (int k = 0; k < n; ++k) {
    for (int i = 0; i < n; ++i) {
        if (dist[i][k] == INF) continue;
        for (int j = 0; j < n; ++j) {
            if (dist[k][j] == INF) continue;
            dist[i][j] = min(dist[i][j], dist[i][k] + dist[k][j]);
        }
    }
}

vector<int> odd;
odd.reserve(n);
for (int v = 0; v < n; ++v) {
    if (degree[v] % 2 == 1) odd.push_back(v);
}

int k = (int)odd.size();
if (k == 0) {
    cout << sumEdges << '\n';
    return 0;
}

int fullMask = (1 << k) - 1;
vector<int> dp(1 << k, INF);
dp[0] = 0;

```

```

for (int mask = 0; mask <= fullMask; ++mask) {
    if (dp[mask] == INF) continue;

    int first = -1;
    for (int i = 0; i < k; ++i) {
        if ((mask & (1 << i)) == 0) {
            first = i;
            break;
        }
    }

    if (first == -1) continue;

    for (int j = first + 1; j < k; ++j) {
        if (mask & (1 << j)) continue;

        int newMask = mask | (1 << first) | (1 << j);
        int add = dist[odd[first]][odd[j]];
        dp[newMask] = min(dp[newMask], dp[mask] + add);
    }
}

int answer = sumEdges + dp[fullMask];
cout << answer << '\n';

return 0;
}

```

4. Оценка временной и пространственной сложности

Пусть:

- n — число вершин,
- m — число рёбер,
- k — число нечётных вершин.

Время:

- чтение графа и подготовка — $O(n + m)$,
- Флойд–Уоршелл — $O(n^3)$,
- DP по маске — $O(2^k \cdot k^2)$.

Итоговая асимптотика по времени:

$$O(n^3 + 2^k k^2).$$

Память:

- матрица расстояний — $O(n^2)$,
- массивы степеней/нечётных вершин — $O(n)$,

- DP — $O(2^k)$.

Итоговая асимптотика по памяти:

$$O(n^2 + 2^k).$$

5. Пример (увеличенный граф)

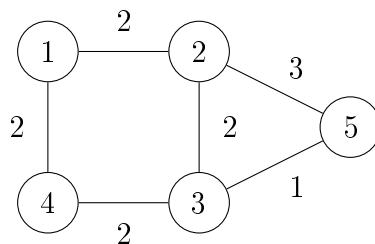
Вход:

```
5 6
1 2 2
2 3 2
3 4 2
4 1 2
2 5 3
3 5 1
```

Выход:

14

Визуализация графа:



Пояснение:

- Сумма всех рёбер: $S = 2 + 2 + 2 + 2 + 3 + 1 = 12$.
- Степени вершин: $\deg(1) = 2$, $\deg(2) = 3$, $\deg(3) = 3$, $\deg(4) = 2$, $\deg(5) = 2$. Нечётные вершины: 2 и 3.
- Кратчайшее расстояние между ними: $\text{dist}(2, 3) = 2$.
- Минимальная добавка: 2.
- Ответ: $12 + 2 = 14$.